

Detecting Recombination in Bacterial Genomes

Module 1

Training Workbook

Overview of Workbook:

Bacteria do not evolve only by accumulating point mutations. They also swap chunks of DNA with other strains through recombination. Those imported chunks carry many SNPs at once, in a burst, at a single point in the tree. If you build a phylogeny without accounting for them, recombination inflates branch lengths, distorts topology, and can make unrelated isolates look like a tight cluster.

In this training, you will build recombination-aware core-genome processes, visualize and quantify the recombination in your Diarrhoeagenic *E. coli* datasets using the following tools:

1. Mash: To screen public reference genomes and pick the one closest to your isolates.
2. Snippy: To map reads to that reference, call variants, build a core-genome alignment.
3. Gubbins: Used to detect and mask recombinant regions, build a clonal-frame tree.
4. IQ-TRE: To build a maximum-likelihood tree from the recombination-filtered SNPs.
5. Bakta: Annotate the reference for gene context.
6. Phandango: View recombination blocks against the tree and genome, interactively.
7. R / ggplot2: Quantify and compare recombination parameters across lineages.

Note: No bioinformatics tool is universally considered "most correct." Every tool has its own strengths and limitations. The tools presented in this workbook have been widely published, validated, and used in research and public health applications. Users should understand each tool's assumptions and limitations when interpreting results.

Open a tmux session before you start

```
tmux new -s mysession
```

Other useful tmux command

```
# List available session
tmux ls

# Attach to a named session
tmux attach -t mysession

# Leave a session without stopping it
ctrl-b d

# Kill running session
tmux kill-session -t mysession
```

Your expected project directory layout on the server

```
clusterXX/
├── fastqs/           # INPUT: paired-end reads e.g., SAMPLE_1.fastq.gz & SAMPLE_2.fastq.gz
├── fastas/           # INPUT: assembled genomes of those same reads (SAMPLE.fasta)
└── recombination/    # OUTPUT: everything this workbook produces lives here
    ├── mash/         # reference selection
    ├── snippy/        # mapping + core alignment
    ├── gubbins/       # recombination masking
    ├── iqtree/        # final tree
    └── bakta/         # reference annotation
```

```
cp -r /data/assigned_cluster/ /path/to/project/home/directory/

cd assigned_cluster/

mkdir -p recombination/{mash,snippy,gubbins,iqtree,bakta}
```

Activate a Conda Environment

```
# Activate the main environment (run this first, in each new terminal)
# source /opt/miniconda3/etc/profile.d/conda.sh
conda activate degain_recomb

# Confirm the tools are on your PATH
which mash snippy run_gubbins.py
```

1.1 What Mash does, and why we start here

Snippy needs one reference genome to map reads against, and the quality of the whole analysis depends on that choice: too distant a reference means reads map poorly, the core genome shrinks, and resolution is lost. So, the first job is to find, among all published complete genomes of your species, the one closest to your isolates.

Mash makes this fast with MinHash sketching: it reduces each genome to a small “sketch” of hashed k-mers (short DNA words), then estimates the Mash distance between two sketches — an approximation of nucleotide divergence ($\approx 1 - \text{ANI}$). **For choosing a reference, smaller distance = closer = better.**

1.2 The script

Step 1 Copy and save script

Copy the content in the pink text box. Save this as `find_closest_reference.sh` inside `recombination/mash/`. The numbered comments trace the strategy; the following sections explain how to run it and how to read its output.

FYI: Downloading all reference genomes for your species of interest (in this case “*Escherichia coli*”) can be time-consuming and may require substantial disk space. Therefore, you do **not** need to run this script for this course. The required reference genomes have already been downloaded and are available in the **XXXXXX** directory. However, it is important to understand how to retrieve these genomes yourself.

```
cd mash/                                # cd into mash directory

nano find_closest_reference.sh           # create file to add script

# Copy and paste the text from the pink box below, then press ctrl + X
# press Y to save

# Do this to run the script
bash find_closest-reference.sh
```

```

#!/usr/bin/env bash
set -euo pipefail

# Screen ../../fastas against every NCBI RefSeq "Complete Genome" of a species
# with Mash; report the reference closest (on average) to the queries.

# Config (edit these as necessary)
CONDA_SH="/opt/miniconda3/etc/profile.d/conda.sh" # from: conda info --base
CONDA_ENV="degain_recomb"
SPECIES="Escherichia coli" # species to screen against
QUERY_DIR="../../fastas" # your assembled genomes
THREADS=8

source "$CONDA_SH"
conda activate "$CONDA_ENV"

# 1. RefSeq assembly summary
if [[ ! -f assembly_summary.txt ]]; then
    wget -q "https://ftp.ncbi.nlm.nih.gov/genomes/refseq/bacteria/assembly_summary.txt"
fi

# 2. Filter to species -> Complete Genome + latest -> build download URLs
grep -F "$SPECIES" assembly_summary.txt > genomes.txt
awk -F '\t' '$12=="Complete Genome" && $11=="latest" {print $20}' genomes.txt > complete_dirs.txt
awk 'BEGIN{FS="/"} {dir=$0; sub(/\//+$/, "", dir); n=split(dir,a,"/"); asm=a[n]; print
dir"/"asm"_genomic.fna.gz" }' complete_dirs.txt > complete_urls.txt
echo "Found $(wc -l < complete_urls.txt) complete genome(s) for $SPECIES"

# 3. Download references and sketch them into ONE combined sketch
mkdir -p refs
wget -q -i complete_urls.txt -P refs || true

echo "Downloaded $(ls refs/*.fna.gz 2>/dev/null | wc -l) of $(wc -l < complete_urls.txt) genomes"

mash sketch -p "$THREADS" -o ref_genomes refs/*.fna.gz

# 4. Sketch each query and measure distance to every reference
for q in "$QUERY_DIR"/*.f*a; do
    name=$(basename "$q%.*")
    mash sketch -p "$THREADS" -o "$name" "$q"
    mash dist "${name}.msh" ref_genomes.msh > "${name}.mash.dist.tsv"
done

# 5. Rank references by MEAN distance across all queries
cat *.mash.dist.tsv > all_distances.tsv
{ printf 'reference\tavg_distance\n'
  awk -F '\t' '{sum[$2]+=$3; n[$2]++}
    END {for (r in sum) printf "%s\t%.6f\n", r, sum[r]/n[r]}' all_distances.tsv \
  | sort -t'\t' -k2,2g
} > av_dist_per_ref.tsv
echo "Top references :"; head av_dist_per_ref.tsv

# 6. Copy the closest reference, unzip, keep only the chromosome
best=$(awk 'NR==2{print $1}' av_dist_per_ref.tsv)
cp "$best" top_ref_genome.fna.gz
gunzip -f top_ref_genome.fna.gz
awk '/^>/{n++} n==1' < top_ref_genome.fna > reference.fna
echo "Reference ready: $(grep -c '^>' reference.fna) sequence in reference.fna"

# 7. Tidy up (keep reference.fna, av_dist_per_ref.tsv, all_distances.tsv)
rm -f /*.msh /*.mash.dist.tsv genomes.txt complete_dirs.txt complete_urls.txt top_ref_genome.fna
rm -rf refs

```

How to read a mash dist table (<isolate>.mash.dist.tsv per isolate.)

Column	Example	Meaning
1 query	isolateA	Your isolate.
2 reference	refs/GCF_...fna.gz	The candidate reference.
3 distance	0.004312	Mash distance $\approx 1 - \text{ANI}$. Smaller = more similar.
4 p-value	0	Chance the match is random; ~0 for real hits.
5 shared	842/1000	Shared hashes / sketch size; higher = stronger.

2. Snippy: Variant Calling & Core Alignment

2.1 What Snippy does

Snippy is a rapid haploid variant caller for bacteria. For each isolate it aligns reads to the reference (bwa-mem), calls SNPs and small indels (FreeBayes), and reports them. snippy-core then merges every isolate's call into a core-genome alignment. The helper snippy-multi generates all per-sample commands plus the core step from a tab-separated sample sheet.

2.2 Running Snippy, step by step

Step 1: Move into the snippy folder and bring in the reference

```
cd snippy                # from recombination/mash
cp ../mash/reference.fna .
```

Step 2: Find the absolute path to your reads

```
cd ../../fastqs && pwd    # copy the path it prints
cd -                     # return to recombination/snippy
```

`pwd` prints the full path of the fastqs folder. Snippy records absolute read paths, so using the full path avoids broken links later.

Step 3: Generate the list of sample IDs

```
FASTQ_DIR="/paste/the/path/from/step2/fastqs"

for f in "$FASTQ_DIR"/*_1.fastq.gz; do
    b=$(basename "$f")
    echo "${b%_1.fastq.gz}"
done > sample_ids.txt

cat sample_ids.txt
```

The loop looks at every R1 file, strips the directory and the _1.fastq.gz suffix, leaving a clean sample name.

Output: sample_ids.txt is a one sample name per line file.

Step 4: Build the snippy-multi sample sheet

```
while read -r id; do
    printf '%s\t%s\t%s\n' "$id" "$FASTQ_DIR/${id}_1.fastq.gz" "$FASTQ_DIR/${id}_2.fastq.gz"
done < sample_ids.txt > samples.txt

cat -A samples.txt | head    # tabs must show as ^I
```

Snippy-multi needs three columns per sample — ID, R1, R2 — separated by real TABs. `printf '%s\t%s\t%s\n'` guarantees genuine tabs; `cat -A` reveals them as `^I`.

NOTE

The most common Snippy failure is a space-separated sample sheet. Verify with `cat -A samples.txt` before Step 5. Do not build it with `sed 's/.../\t/'` — not every `sed` turns `\t` into a real tab.

Step 5: Generate the Snippy commands, inspect, then run

```
snippy-multi samples.txt --ref reference.fna --cpus 16 > run_snippy.sh

# Look at what will run BEFORE running it (press q to exit)
less run_snippy.sh

# Execute the per-sample calling + the core step
bash run_snippy.sh
```

snippy-multi writes a command list to `run_snippy.sh`: one snippy line per isolate then a snippy-core line.

Output: one folder per isolate plus the `core.*` files below.

What each isolate's folder contains

File	What it contains
snps.tab	Readable table of every variant: position, type, ref & alt alleles, effect.
snps.vcf	The same variants in standard VCF format.
snps.bam	Reads aligned to the reference (you can open in IGV to inspect a call).
snps.aligned.fa	Reference with this isolate's alleles; low coverage N, zero coverage .
snps.txt	Run summary: reads, coverage, variant counts.

File	What it is	Used for
core.full.aln	Whole-genome alignment (all samples + reference), including invariant sites .	Input to Gubbins.
core.aln	Core SNP alignment (variable sites only), FASTA.	A quick tree without masking.
core.tab	Core SNP positions and each sample's allele.	Eyeballing SNPs.
core.vcf	Multi-sample VCF of core variants.	Downstream VCF tools.
core.txt	Per-sample coverage summary.	QC to spot poor isolates.

Step 6: Clean the alignment for Gubbins

```
snippy-clean_full_aln core.full.aln > clean.full.aln
```

snippy-clean_full_aln converts uncertain characters (N, -) into a form Gubbins accepts. Required before Gubbins.

Output: `clean.full.aln` — the cleaned whole-genome alignment, ready for recombination masking.

3. Gubbins : Detecting & Masking Recombination

3.1 What Gubbins does, and why it matters

Gubbins finds regions with an unusually high density of SNPs on a single branch . This is the signature of an imported, recombinant chunk. It iterates build a tree, scan each branch for dense SNP clusters, flag them as recombination, remove them, rebuild, and repeat until stable. The result is a phylogeny built only on vertically inherited variation.

NOTE: Gubbins needs a whole-genome alignment with several isolates, not a SNP-only alignment. That is why we feed it `clean.full.aln`, which keeps the invariant sites.

3.2 Running Gubbins, step by step

Step 1: Run recombination detection

```
run_gubbins.py --prefix ../gubbins/gubbins --threads 16 clean.full.aln
```

Runs the full iterative analysis. `--prefix ../gubbins/gubbins` writes outputs into `gubbins/` with names starting `gubbins`; `--threads 36` sets CPUs. An explicit prefix keeps filenames predictable for the next sections.

Useful extra options:

- `--tree-builder iqtree` choose the internal tree method.
- `--filter-percentage 25` drop taxa with >25% missing data.
- `--iterations 5` cap the number of refinement cycles.

What Gubbins produces

File (prefix <code>gubbins.</code>)	What it is	Why you care
<code>filtered_polymorphic_sites.phylip</code>	Recombination-filtered SNP alignment (Phylip).	Input to IQ-TREE.
<code>filtered_polymorphic_sites.fasta</code>	Same alignment in FASTA.	Alternative input.
<code>recombination_predictions.gff</code>	Every predicted recombinant region (GFF3): start, end, branch.	Load into Phandango.
<code>recombination_predictions.embl</code>	Same predictions in EMBL format.	Viewing in Artemis.
<code>final_tree.tre</code>	Clonal-frame tree (recombination removed), Newick.	Load into Phandango / IQ-TREE.
<code>node_labelled.final_tree.tre</code>	Final tree with internal nodes labelled.	Cross-reference with the stats CSV.
<code>per_branch_statistics.csv</code>	Per-branch SNP counts, blocks, r/m , ρ / θ .	The data for Section 8 (R).
<code>summary_of_snp_distribution.vcf</code>	Where SNPs sit relative to recombinant regions.	Site-level inspection.

Step 2: Inspect the per-branch recombination statistics

```
column -t -s, ../gubbins/gubbins.per_branch_statistics.csv | less -S
```

Opens the QC table with columns aligned (`-s`, splits on commas; `less -S` stops long lines wrapping). Each row is one branch. The two you will quote most:

- SNPs Inside / Outside Recombinations: how many of the branch's SNPs came from imported chunks (r) versus point mutation (m).
- r/m : recombination SNPs $\div$ mutation SNPs.
 $r/m > 1$ = recombination contributed more variation than mutation on that branch; $r/m \approx 0$ is near-clonal.

TIP: Report the genome-wide r/m — the standard one-number summary of how much recombination is occurring, and your justification for masking it.

4. IQ-TREE: The Recombination-Free Tree (Optional)

Gubbins builds a fast internal tree, but for the final phylogeny we hand its recombination-filtered SNP alignment to IQ-TREE, a maximum-likelihood builder with rich models and branch support.

4.1 Running IQ-TREE

Step 1: Build the maximum-likelihood tree

```
conda activate degain_iqtree3
cd iqtree
iqtree3 -s ../gubbins/gubbins.filtered_polymorphic_sites.phylip \
```

Flags:

- `-s ...phylip` the Gubbins recombination-filtered SNP alignment.
- `-m MFP` to auto-select the best model.
- `-nt AUTO` auto-detect thread count.
- `-pre ../iqtree/core_tree` write outputs into iqtree/ with prefix `core_tree`.
- `-bb 1000 1000` ultrafast bootstrap replicates for branch support.

Output:

File	What it is
<code>core_tree.treefile</code>	The final ML tree in Newick. Can be open in FigTree, iTOL, or ggtree.
<code>core_tree.iqtree</code>	Full readable report: model, likelihood, parameters.
<code>core_tree.contree</code>	Consensus tree with bootstrap support (from <code>-bb</code>).
<code>core_tree.log</code>	Run log, check here if something fails.

5. Bakta: Annotating the Reference

Bakta annotates the reference genome so that when you find a recombinant region or SNP you can say which gene it falls in.

5.1 Running Bakta, step by step

Step 1: Activate Bakta environment

```
conda activate degain_bakta
```

Step 2: Move to the recombination folder

```
cd ../clusterXX/recombination/
```

Step 3: Run the annotation

```
bakta --db /home/okeke/local_data/bioinfo/db \
--output bakta/reference \
--prefix reference \
--threads 4 \
--complete --skip-ori --force \
snippy/reference.fna
```

Flag by flag:

- `--db /path/to/bakta_db` the downloaded Bakta database (the data, not the software — download once, it is large).
- `--output bakta/reference` results directory. Use a relative path — **not** an absolute `/bakta_results`, which starts at the filesystem root and fails with a permission error.
- `--prefix reference` base name for the output files.
- `--threads 36` CPUs to use.
- `--complete` the reference is a finished genome.
- `--skip-ori` skip origin-of-replication detection (faster).
- `--force` overwrite an existing output folder.
- `snippy/reference.fna` the genome to annotate.

What Bakta produces (in bakta/reference/)

File	What it is
reference.gff3	Annotations in GFF3. The gene track you load into Phandango and intersect with recombination blocks.
reference.gbff	GenBank-format annotated genome.
reference.faa	Predicted protein sequences (amino acids).
reference.ffn	Nucleotide sequences of predicted genes.
reference.tsv	Tabular feature list
reference.txt	Summary counts (CDS, tRNA, rRNA, etc.).

6. Viewing Recombination Blocks in Phandango

7.1 What Phandango is

Phandango is a free, browser-based interactive viewer for bacterial population genomics. There is nothing to install. You open a web page and drag your files onto it. It lines the phylogeny up against the length of the reference genome and draws the Gubbins recombination blocks as coloured bars, so you can see where recombination happened at a glance, and on which branches, and in which genes.

7.2 The files you import

Phandango reconstructs the whole picture from files you already produced. Drag all of them onto the window together in a single drop:

File	From	Required?	What it provides
<code>gubbins.final_tree.tre</code>	Gubbins	Yes	The phylogeny on the left (Newick).
<code>gubbins.recombination_predictions.gff</code>	Gubbins	Yes	The recombination blocks drawn against the genome (GFF3).
<code>reference.gff3</code>	Bakta	Recommended	A gene annotation track so you can read which genes a block covers. Must span the same length as the alignment/reference.
<code>metadata.csv</code>	You	Optional	Coloured strips beside each tip (e.g. lineage, source, date) — simple CSV, first column = tip name.

7.3 Loading the data, step by step

Step 1: Open Phandango

```
# In your web browser, go to:  
https://jameshadfield.github.io/phandango/
```

You will see a mostly empty page with instructions to drop files.

Step 2: Select the files together

In your file manager, select the three (or four) files at once: `gubbins.final_tree.tre`, `gubbins.recombination_predictions.gff`, `reference.gff3`, and optionally `metadata.csv`.

Step 3: Drag and drop

Drag the selected files onto the Phandango window and release. Phandango detects each file type automatically and renders the combined view.

7.4 Reading the display

The view has two linked panels. On the left is the Gubbins tree; on the right is a horizontal track representing the reference genome from position 1 to its end. Each row on the right lines up with the tip at the same height on the tree, and each coloured bar is a recombination block at that genomic position:

What you see	What it means
Red bars	Recombination reconstructed on an internal branch. This is an ancestral event, therefore shared by all isolates descending from that branch through common ancestry.
Blue bars	Recombination reconstructed on a terminal branch. This is unique to a single isolate, acquired since it diverged.
A vertical band of blocks hitting many isolates at the same position	A recombination hotspot. A region imported repeatedly across the population. Often a mobile element, prophage, capsule/O-antigen locus, or an antibiotic-resistance region.
Blocks concentrated on one clade only	Lineage-specific recombination. That group has taken up DNA the others have not, a clue to distinct ecology or selection.
The annotation track lined up under a block	The gene(s) the recombination covers. Click/zoom to read names.

7.5 What it tells you about the genomes

Reading the pattern as a whole is where the biology comes out:

- Lots of **red** (deep, shared) blocks means recombination is old and largely ancestral. Could mean the population diversified through that import long ago.
- Lots of **blue** (tip-specific) blocks means recombination is ongoing. Here isolates are actively acquiring DNA.
- Vertical hotspots shared across lineages flag regions under strong selection (resistance, immune-evasion, surface antigens). These are prime candidates to investigate, and regions you were right to mask before building the tree.
- A near-empty genome track means a near-clonal population where the unmasked SNP tree would already have been reliable.

7. Quantifying Recombination Parameters in R

Phandango shows you the recombination; R lets you **measure and compare** it. In this section you install R and RStudio, read Gubbins' `per_branch_statistics.csv`, and draw box plots comparing recombination parameters such as r/m , recombination block length, and block count across three lineages.

8.1 Installing R and RStudio (Optional)

Install R first, then RStudio. These are two separate pieces:

R is the language/engine;

RStudio is the friendly editor you work in.

Step 1: Install R (the engine)

- Windows / macOS: download the installer from <https://cran.r-project.org> and run it (accept the defaults).

Step 2: Install RStudio Desktop (the editor)

Download RStudio Desktop (free) from <https://posit.co/download/rstudio-desktop/> and install it. Open RStudio, it finds R automatically.

Step 3: Open the R script provided and run it

Questions:

1. Across most branches, are there more SNPs inside or outside recombination regions (Figure 1), and what does that imply about the dominant source of genetic diversity in these lineages?
2. Which lineage is the most recombinogenic, and how do you know?
3. r/m measures SNPs imported by recombination relative to those from mutation. If a lineage's median r/m sits below 1, what are you concluding biologically and why is the *median* near zero even when the mean (red dot) is higher?